

SESSION-BASED vs JWT

En la actualidad, el manejo de la información requiere de mecanismos de verificación para garantizar que la comunicación entre cliente y servidor sea segura y confiable. Con el crecimiento de las aplicaciones web, las APIs REST, la pregunta de cómo autenticar y autorizar a los usuarios de forma eficiente se ha convertido en uno de los temas fundamentales del diseño de software. En este contexto, dos enfoques han dominado el panorama del desarrollo web: la autenticación basada en sesiones (Session-Based Authentication) y la autenticación mediante JSON Web Tokens (JWT). Ambas estrategias persiguen el mismo objetivo que es verificar la identidad del usuario y controlar su acceso a los recursos, pero lo hacen de maneras distintas, cada una con sus propias ventajas y desventajas.

Para comprender mejor las diferencias es necesario partir desde la parte conceptual. Por un lado, la autenticación basada en sesiones se apoya en el paradigma stateful (Con Estado), En este modelo tanto el cliente como el servidor mantienen información activa sobre la sesión del usuario así cuando alguien inicia sesión, el servidor genera un identificador único que almacena en su memoria o base de datos, y que también entrega al cliente generalmente mediante una cookie, para que sea presentado en cada solicitud que el usuario realice. Por otro lado, JWT implementa el paradigma stateless (Sin estado), pues toda la información necesaria para autenticar al usuario viaja codificada dentro del propio token, eliminando la necesidad de que el servidor guarde ningún registro de sesión activa.

Un elemento adicional que tenemos que agregar es el protocolo HTTP sobre el cual operan ambos métodos de autenticación. HTTP es un protocolo sin memoria lo que significa que cada solicitud que el cliente envía al servidor es completamente independiente de las anteriores, y es por este diseño que el servidor no tiene manera de saber quién realizó la petición ni qué operaciones ha ejecutado un usuario durante su enlace. Esta característica hace que la autenticación no sea simplemente una buena práctica, sino una necesidad arquitectónica. Cada solicitud debe incluir suficiente información para que el servidor pueda validar la identidad del solicitante y decidir si

debe o no conceder acceso al recurso solicitado. Tanto Session-Based como JWT son respuestas directas a este problema, aunque lo resuelven de manera distinta.

Al momento de seleccionar un mecanismo de autenticación, los arquitectos de software se enfrentan a lo que puede llamarse como el trilema de la autenticación, la tensión permanente entre seguridad, experiencia del usuario y rendimiento del sistema. Este trilema reconoce que maximizar los tres vértices simultáneamente es extremadamente difícil, pues los avances en uno de ellos tienden a comprometer a alguno de los otros. Un sistema que prioriza la seguridad al máximo puede volverse lento y frustrante para el usuario dependiendo de cómo se maneje la actualización del token. Uno que priorice la experiencia del usuario podría descuidar controles de seguridad que puede abrir la puerta a vulnerabilidades. Y uno que busca rendimiento extremo podría sacrificar verificaciones que comprometan la seguridad. Tanto Session-Based como JWT representan distintas posiciones dentro de este trilema, y entender dónde se ubica cada uno permite tomar decisiones de diseño más certeras.

La autenticación basada en sesiones opera de la siguiente manera: cuando un usuario proporciona sus credenciales y estas son validadas y el servidor genera un identificador de sesión que almacena en su propia memoria, una base de datos relacional o un almacén de clave-valor como Redis. Este identificador se entrega al cliente como cookie, y en cada solicitud posterior el navegador lo incluye automáticamente para que el servidor lo busque y confirme que corresponde a una sesión activa. Una ventaja importante de este esquema es el control que ofrece sobre la sesión, el servidor puede revocar el acceso en cualquier momento eliminando el registro almacenado. Este nivel de control es valioso en aplicaciones donde la seguridad es crítica, como plataformas bancarias, donde invalidar un acceso de forma instantánea ante una actividad sospechosa puede ser determinante.

JWT adopta un enfoque diferente. Al autenticarse el servidor no guarda ningún estado en cambio genera un token que entrega al cliente compuesto por tres partes separadas por puntos: el header que indica el tipo de token y el algoritmo de firma, el payload que contiene la información del usuario como su identificador, roles y tiempo de expiración y la signature que permite verificar que el token no fue alterado desde su emisión. El servidor puede autenticar

cualquier solicitud simplemente verificando la firma, sin necesidad de consultar ninguna base de datos ni registro centralizado.

En cuanto a la seguridad cada mecanismo presenta sus propios riesgos. En el caso de las sesiones las amenazas más relevantes son el secuestro de sesión y el Cross-Site Request Forgery (CSRF). El CSRF es una vulnerabilidad que engaña a usuarios autenticados para que envíen solicitudes no deseadas, explotando el manejo de sesiones basado en cookies para ejecutar acciones no autorizadas en nombre del usuario sin su conocimiento. Para mitigar estos riesgos, se recomienda el uso de cookies con el atributo `HttpOnly`, que impide que JavaScript acceda a la cookie, reduciendo el riesgo de que sea robada mediante XSS, y el atributo `Secure`, que garantiza que la cookie solo se transmita sobre HTTPS.

JWT enfrenta un riesgo diferente que sería el robo del token. La seguridad de los JWT depende en gran medida de cómo y dónde se almacenan en el cliente, ya que un almacenamiento inadecuado puede exponer estos tokens al robo y al uso indebido. Cuando los tokens se guardan en `localStorage` del navegador quedan vulnerables a ataques XSS, pues cualquier script malicioso que se ejecute en la página puede leerlos y extraerlos. La estrategia considerada más robusta es mantener el access token de vida corta en memoria de la aplicación y el refresh token en una cookie `HttpOnly` segura, lo que provee la mejor defensa posible contra XSS y CSRF simultáneamente. Adicionalmente, JWT presenta un reto particular a diferencia de las sesiones un token emitido permanece válido hasta su expiración aunque el usuario haya cerrado sesión. Para acotar este problema, las implementaciones modernas combinan tokens de acceso de vida corta con tokens de refresco, limitando la ventana de exposición ante un token comprometido.

Finalmente uno de los factores donde las diferencias entre ambos métodos se hacen más evidentes es en la escalabilidad horizontal que es la capacidad de distribuir la carga entre múltiples servidores en paralelo. La autenticación basada en sesiones crea el problema de las sticky sessions cuando el usuario se autentica contra un servidor específico, su información de sesión vive en esa instancia, obligando al balanceador de carga a dirigir todas sus solicitudes futuras hacia ese mismo servidor, lo que introduce complejidad y potenciales puntos únicos de falla. La alternativa es centralizar las sesiones en un repositorio compartido como Redis, pero esto agrega latencia y complejidad de infraestructura. JWT elimina este problema porque no requiere un almacén central de sesiones, cada servidor verifica la firma del token de forma

independiente permitiendo que el balanceador de carga distribuya las solicitudes libremente sin restricciones de afinidad, lo que facilita agregar instancias nuevas cuando el tráfico aumenta.

Por lo tanto podemos decir que Session-Based y JWT son dos respuestas válidas al reto que impone la naturaleza sin estado de HTTP, y cada una ocupa una posición distinta dentro del trilema entre seguridad, experiencia y rendimiento. Las sesiones ofrecen mayor control y mayor complejidad en entornos distribuidos mientras que JWT entrega escalabilidad y simplicidad arquitectónica a cambio de ceder parte del control de vida del token. La clave no está en elegir el mejor método, sino en entender qué exige el contexto de cada aplicación y aplicar las contramedidas adecuadas sobre el que se adopte.

Javier Isaías Coyoy Marín
202030556

Bibliografía

- https://assets.ctfassets.net/2ntc334xpx65/20qmpOMjITMY8YCBxGmnsd/5ebe3c898dd69d95f87efacb732ef302/Auth0_JWT_Handbook_ES.pdf.
- https://assets.ctfassets.net/2ntc334xpx65/o5J4X472PQUI4ai6cAcqg/13a2611de03b2c8edbd09c3ca14ae86b/jwt-handbook-v0_14_1.pdf
- <https://blog.logto.io/es/autenticacion-basada-en-tokens-vs-autenticacion-basada-en-sesion>
- <https://es.scribd.com/document/643305993/Gestion-de-Sesiones-pdf>
- <https://es.scribd.com/document/745446211/tema-5-gestion-de-sesiones-y-autorizacion-en-aplicaciones-web>